

Recap 9th grade - II

Elementary Algorithms in C++

1. Digits of a Number

Extracting and processing digits in C++

Processing Digits

Any number can be dismantled digit by digit using the **modulo (%)** and **division (/)** operators.

```
// Sum of the digits of a number
int n, sum = 0;
cin >> n;

while (n > 0) {
    int c = n % 10; // Extract last digit
    sum += c;       // Process the digit
    n = n / 10;     // Remove last digit
}

cout << "Sum of digits: " << sum;
```



2. GCD and LCM

Euclidean Algorithm

Euclidean Algorithm (GCD)

Repeated Division

The most efficient method using the modulo operator until the remainder is zero.

```
int a, b;  
cin >> a >> b;  
  
while (b != 0) {  
    int r = a % b;  
    a = b;  
    b = r;  
}  
  
// 'a' stores the GCD
```

Repeated Subtraction

The classic method subtracting the smaller number from the larger number repeatedly.

```
int a, b;  
cin >> a >> b;  
  
while (a != b) {  
    if (a > b)  
        a = a - b;  
    else  
        b = b - a;  
}  
  
// 'a' (or 'b') stores the GCD
```

Least Common Multiple (LCM)

The Formula

Once the Greatest Common Divisor (GCD) is calculated, the LCM is obtained via a direct mathematical formula:

$$\text{LCM} = \frac{a \cdot b}{\text{GCD}}$$

C++ Implementation

We must save the original product before modifying `a` and `b` to find the GCD.

```
int a, b;  
cin >> a >> b;  
int prod = a * b;  
  
// Calculate GCD in 'a' here ...  
  
int lcm = prod / a;  
cout << "LCM: " << lcm;
```

3. Prime Numbers

Optimizing the primality test

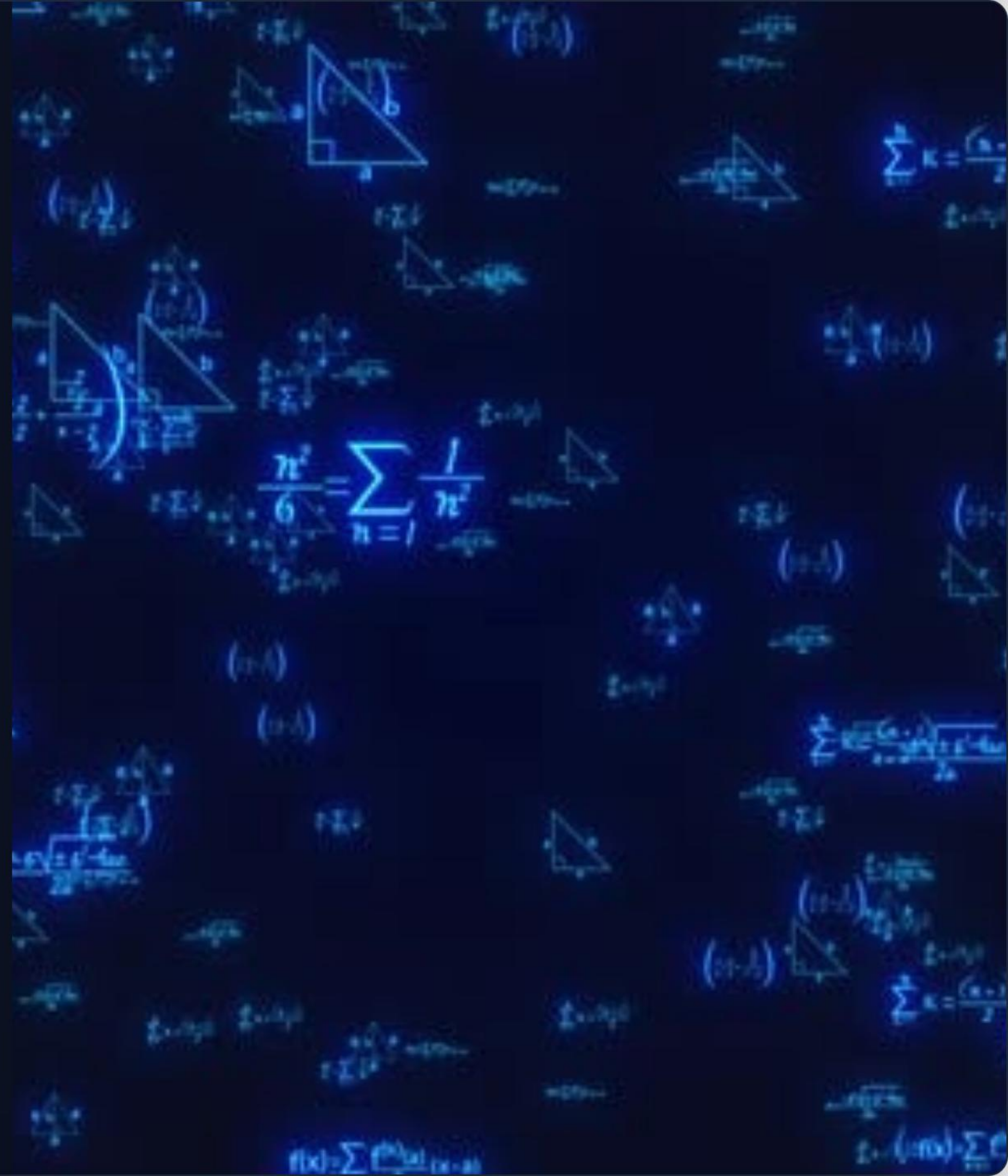
Checking Primality

A number `n > 1` is prime if it has exactly two divisors (1 and itself). There is no need to search for divisors all the way up to `n-1`.

We can optimize the algorithm by checking potential divisors only up to the **square root** of the number (`d \* d <= n`).

```
bool is_prime = true;
if (n < 2) is_prime = false;

for (int d = 2; d * d <= n; d++) {
    if (n % d == 0) {
        is_prime = false;
        break;
    }
}
```



4. Prime Factorization

The Fundamental Theorem of Arithmetic

Factorization Steps



1. Divide

Start with the first possible prime divisor ($d = 2$). While n is divisible by d , actively divide it ($n /= d$) and count the power of the factor.



2. Advance

When n is no longer divisible by d , move to the next potential divisor ($d++$). Continue this process as long as $n > 1$.



3. Optimize

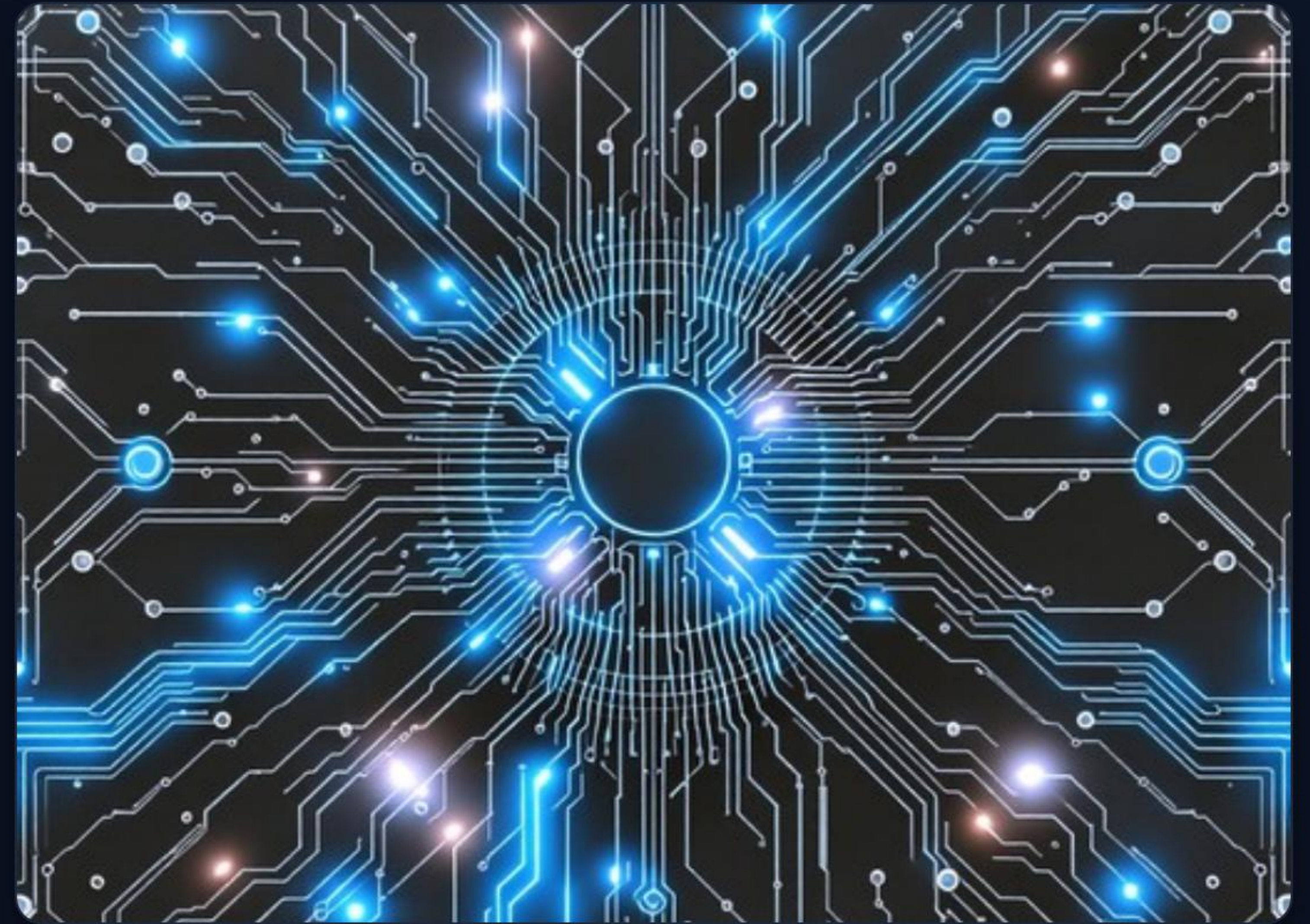
Similar to prime numbers, if $d * d > n$ and $n > 1$, we can instantly deduce that the remainder of n is itself a prime factor.

Implementing Factorization

```
// Fast factorization in C++
int n, d = 2;
cin >> n;

while (n > 1) {
    int p = 0;
    while (n % d == 0) {
        p++;
        n /= d;
    }
    if (p > 0)
        cout << d << "^" << p << " ";

    d++;
    if (d * d > n) d = n; // Optimize
}
```



5. Factorization Applications

Number properties and formulas

Divisors of a Number

Number of Divisors

Obtained by adding 1 to the exponents of all prime factors and multiplying them together.

$$\text{NumDiv} = (e_1 + 1) \cdot (e_2 + 1) \cdot \dots$$

Ex: For $n = 12$ ($2^2 * 3^1$) $\rightarrow (2+1)*(1+1) = 6$ divisors.

Sum of Divisors

Allows calculating the sum of all divisors without actually generating them.

$$\text{Sum} = \frac{p_1^{e_1 + 1} - 1}{p_1 - 1} \cdot \dots$$

A crucial application for quickly solving competitive programming problems.

Image Sources



https://images.stockcake.com/public/0/b/7/0b799ca0-58a1-4c00-aaba-cc04865a12db_medium/digital-data-cube-stockcake.jpg

Source: stockcake.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/0084181b-f79a-409e-9941-4f8fc8c2ba2b/video_preview/video_preview_0002.jpg?w=1200&h=630&cf_fit=crop&q=85&format=jpeg&s=71b02281913b4e4d3980081db003053675b7440474825f1947da510594ccf87c

Source: elements.envato.com



https://images.stockcake.com/public/3/3/e/33e9ba36-6802-42c6-a7b9-83d10bf10461_large/glowing-circuit-network-stockcake.jpg

Source: stockcake.com